

## Relatório 5 - Prática: Construindo um fluxo n8n(II)

Danillo Guimarães

### Descrição da atividade

A partir do docker instalado, instalamos uma ferramenta chamada waha core. Essa instalação permite a utilização de um número/sessão por instância. É possível utilizar versões mais robustas e com capacidade de utilizar até sessões por instância por meio de doações. Veja o comparativo abaixo:

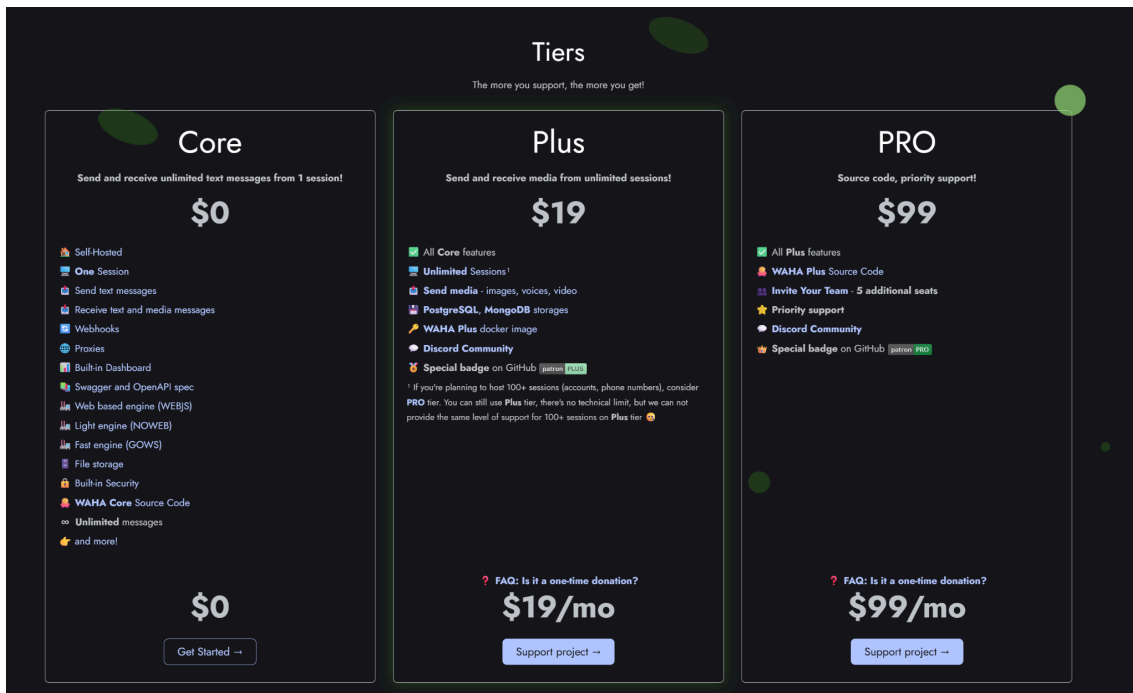


Figura 1 - Planos de utilização do waha que permita facilidade na escala

Para realizar a integração entre o n8n e o servidor waha, é necessário a instalação do plugin de comunidade no n8n do waha e sua posterior configuração. Depois de instalado, o n8n possuirá novos tipos de nodes, justamente os nodes do waha. Na primeira configuração, utilizamos a trigger de inicialização do waha, disparando o trigger de mensagem, passando pelo Message seen (que gera o evento de visualização da mensagem) e posterior resposta da mensagem. Veja a imagem do

canva do n8n.

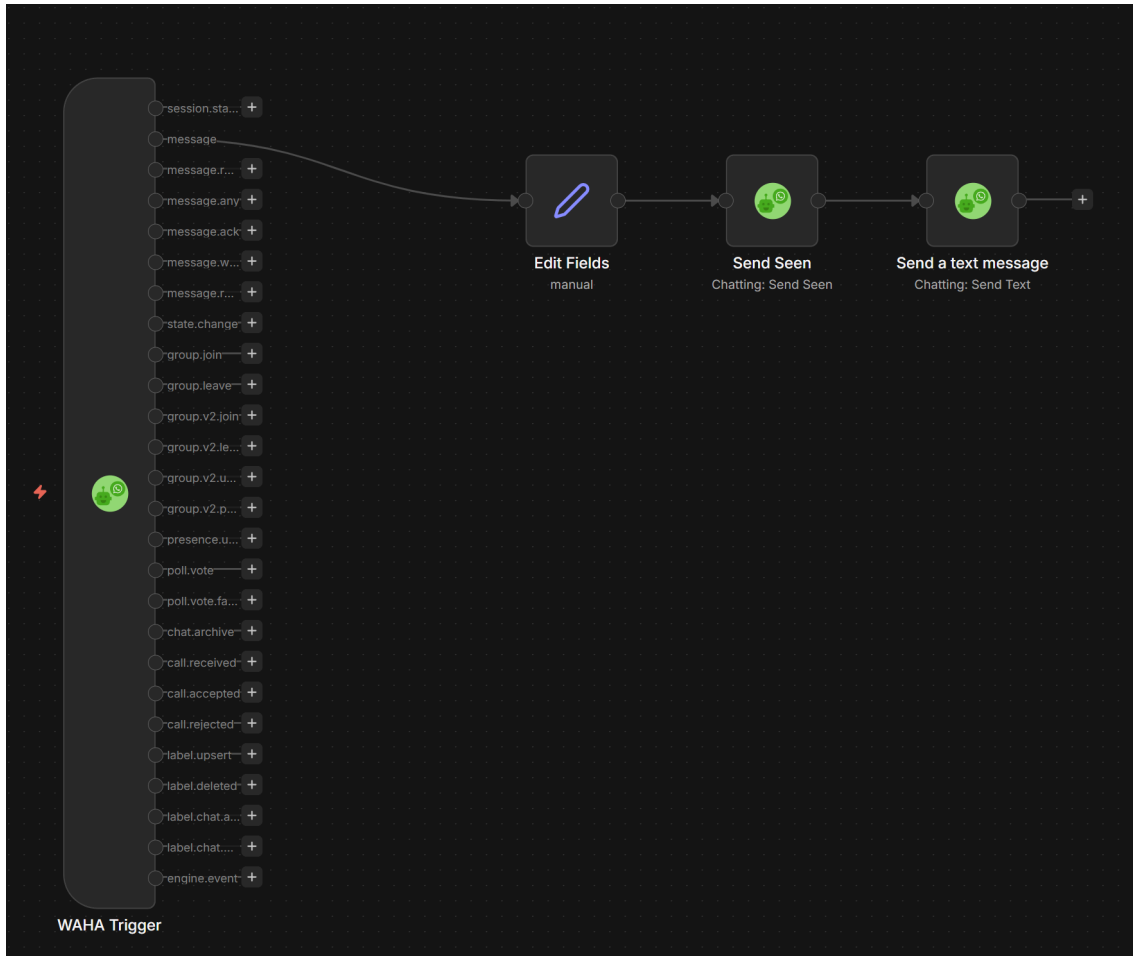


Figura 2 - Fluxo n8n utilizando waha e respondendo mensagens fixas no whatsapp.

A última fonte recomendada pelo card, traz uma orientação em 7 passos para construir um AI Chatbot.

- Step 1: Define your chatbot's purpose
- Step 2: Identify your target audience and use cases
- Step 3: Choose a development platform
- Step 4: Design the conversation flow
- Step 5: Leverage state-of-the-art LLMs for conversational intelligence
- Step 6: Connect to external data sources and channels
- Step 7: Test, refine, and deploy

Por fim realizei uma integração entre os dois últimos materiais. Utilizei o waha pra fazer a integração com uma LLM, no caso um modelo do open AI para responder a um

número pessoal no whatsapp em um servidor remoto..

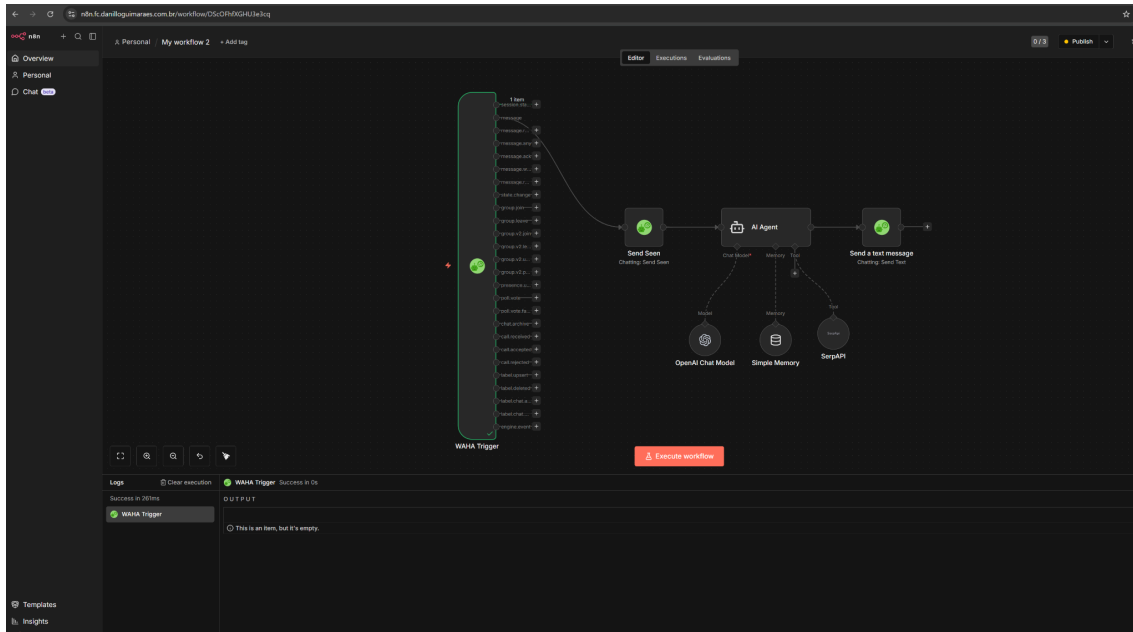


Figura 3 - Fluxo n8n integrando waha com LLM e utilizando uma tool para responder no whatsapp

O objetivo inicial era uma simples consulta no calendário semanal (onde passei via prompt) mas que pretendo integrar com Google Calendar. O público alvo seria eu mesmo.

## Dificuldades

Na versão atual, é necessário utilizar usuário e senha pra acessar o swagger, e posteriormente é necessário utilizar uma api key para conectar o servidor. Isso é uma diferença da versão exibida no vídeo.

```
[16:56:34.818] INFO (Router Explorer/77): Mapped {/api/apps/chatwoot/locales, GET} route
[16:56:34.818] INFO (NestApplication/7): Nest application successfully started
[16:56:33.475] INFO (Bootstrap/7): NODE - Catching unhandled rejections and exceptions enabled
[16:56:33.475] INFO (Bootstrap/7): WAHA (WhatsApp HTTP API) - Running CORE version...
[16:56:34.819] INFO (Bootstrap/7): WhatsApp HTTP API is running on: http://[::1]:3000
[16:56:34.819] INFO (Bootstrap/7): Environment
{"version":"2026.2.1","engine":"NOWEB","tier":"CORE","browser":null,"platform":"linux/x64","worker":{"id":null}}

Generated credentials (persist to .env or WAHA_* env vars)
Save these values to your environment (.env or WAHA_*) to reuse them; new keys are generated on every start otherwise.

cat <<'EOF' >> .env

WAHA_API_KEY=81647cf2822144ffada5962ee483f007
WAHA_DASHBOARD_PASSWORD=bd0d9fe5a2374f71a624442f6d970256
WHATSAPP_SWAGGER_USERNAME=admin
WHATSAPP_SWAGGER_PASSWORD=bd0d9fe5a2374f71a624442f6d970256
EOF

Generated credentials ready to copy
```

Figura 4 - Credenciais expostas no console em caso de não definição no momento do bootstrap

Outra dificuldade é utilizar múltiplas sessões. Para isso é necessário utilizar várias instalações do waha, ou utilizar waha plus, que não é gratuito.

## **Conclusões**

Fazendo um paralelo com o que aprendemos até agora, as capacidades disponibilizadas pelo waha, se tornam as tools do agente ReAct. Nos exemplos nós utilizamos tools para cálculos, aqui o exemplo permite o recebimento da mensagem e o envio de uma resposta.

## **Referencias**

- <https://waha.devlike.pro/support-us/>
- [https://www.youtube.com/watch?v=FyQivMjb3\\_8](https://www.youtube.com/watch?v=FyQivMjb3_8)
- <https://waha.devlike.pro/docs/how-to/install/>
- <https://blog.n8n.io/how-to-make-ai-chatbot/>